Investigating the interaction of linguistic and mathematical reasoning in language models using multilingual number puzzles

Antara Raaghavi Bhattacharya[1, 2], Isabel Papadimitriou[3], Kathryn Davidson[2], David Alvarez-Melis[1,3]

[1]School of Engineering and Applied Sciences, Harvard University
[2]Department of Linguistics, Harvard University
[3]Kempner Institute for the Study of Natural and Artificial Intelligence at Harvard University

Correspondence: antara@alumni.harvard.edu

arXiv:2506.13886v2 [cs.CL] 15 Oct 2025

Abstract

Across languages, numeral systems vary widely in how they construct and combine numbers. While humans consistently learn to navigate this diversity, large language models (LLMs) struggle with linguistic-mathematical puzzles involving cross-linguistic numeral systems, which humans can learn to solve successfully. We investigate *why* this task is difficult for LLMs through a series of experiments that untangle the linguistic and mathematical aspects of numbers in language. Our experiments establish that models cannot consistently solve such problems unless the mathematical operations in the problems are explicitly marked using known symbols ($+$, $\times$, etc, as in "twenty + three"). In further ablation studies, we probe how individual parameters of numeral construction and combination affect performance. While humans use their linguistic understanding of numbers to make inferences about the implicit compositional structure of numerals, LLMs seem to lack this notion of implicit numeral structure. We conclude that the ability to flexibly infer compositional rules from implicit patterns in human-scale data remains an open challenge for current reasoning models.

1 Introduction

Language models reason and solve problems using language. What is the connection (and the integration) between their linguistic systems and their impressive reasoning abilities? To investigate this question, we run a suite of experiments to analyze how language models solve puzzles about diverse linguistic number systems. People represent numbers through language, using rule-based systems that are simultaneously linguistic and mathematical (Ifrah, 2000; Dehaene, 2011; Carey, 2004; Le Corre and Carey, 2007; Ionin and Matushansky, 2006; Hammarström, 2010; Comrie, 2011). Unlike most mathematical reasoning problems, where the mathematical operators are explicit, a numeral system contains implicit operations for describing numerals, and there is considerable variety in how this is done across the world's languages. For example, French *vingt-neuf* (20 + 9), Bengali *untirīsh* (30 − 1), Tamil *irupatti oṉpatu* (2 × 10 + (10 - 1)), and Birom *bākūrū bībā ná vɛ̀ tùŋūn* (2 × 12 + 5) all evaluate to the Hindu-Arabic numeral 29.

We investigate the capabilities of language models to solve puzzles about linguistic number systems, drawn from linguistics competitions (Linguistics Olympiads) where high-school students have to reason through data about unknown languages and explain the linguistic rules governing the data (Derzhanski and Payne, 2010). While language models approach human performance on several language-based benchmarks (Hendrycks et al., 2020; Kojima et al., 2022; Beguš et al., 2023), and recent reasoning models deliberately optimized for logical and mathematical reasoning show remarkable performance improvements for many structured mathematical reasoning tasks (Zhong et al., 2024; Jaech et al., 2024), LLMs perform extremely poorly at solving linguistic-mathematical puzzles about systems of numbers in different languages (Derzhanski and Veneva, 2018; Bean et al., 2024).

Why do language models fail to solve these problems at the intersection of language and math — what specifically causes this failure? And how much of this failure is due to the linguistic vs. the mathematical aspects of the problem?

We present a method to systematically isolate individual parameters of number construction and combination and investigate how they affect language model performance. We establish that most individual mathematical features (like base) do not hinder the ability of sufficiently advanced language models to solve such problems. However, unless **the mathematical operations in a problem are made explicit through familiar symbols (+, ×, etc.)**, models cannot consistently solve the problem. This indicates that, at least within the domain of

Variable	Operator			Example
	Explicitness	Familiarity	Type	
Single character	Implicit	-	-	A B
	Explicit	Familiar	Symbol	A + B
	Explicit	Unfamiliar	Symbol	A α B
	Explicit	Unfamiliar	Word	A *xebrut* B
Multi-character	Implicit	-	-	gbaifi pagig
	Explicit	Familiar	Symbol	gbaifi + pagig
...	...	...	...	

Table 1: A demonstration of the experimental conditions for our explicit operators experiment. We add explicit operators to our base IMPLICIT problems, using both the familiar symbols for addition/multiplication/subtraction, as well as unfamiliar symbols and words to symbolize the operation.

linguistic-mathematical problems, models cannot infer the compositional structure of numerals like humans can, or sufficiently abstract notions like operators. We discuss our findings in the broader context of human language, concluding that flexible, adaptive use of language across domains appears to remain challenging for LLMs.

2 Background

2.1 Linguistic and cognitive connections

People acquire systems of number representation as part of learning language, and are consequently able to construct arbitrary numerals using the rules that they learn. Although the system of rules may be language-specific, the general framework of numeral construction and combination is a fundamental cognitive ability (Hurford, 1987; Feigenson et al., 2004). Performing mathematics in a symbolic sense requires explicit instruction (e.g. a child would not inherently know what + connotes), but once this symbolic meaning has been learned, people can generalize it to apply to any numbers (Sarnecka et al., 2015).

Numeral operations in language can be marked both explicitly (e.g. *und* in German *einundzwanzig*) and implicitly (as in English *twenty-one*), with larger numerals often using a combination of implicit and explicit operations (*five hundred and one* = 5 [$\times$] 100 + 1). Even when operations are implicit, people can understand and infer the cross-linguistic compositional structure of numerals (Ionin and Matushansky, 2006).

In a linguistics contest, a high-school student would not need to know any mathematical concepts beyond basic arithmetic to reason through number system problems and infer the rules needed to solve them. The challenge lies instead in whether models can learn and infer such rules from limited data — a characteristic capacity of humans acquiring language.

2.2 Mathematical ability in language models

Recent language models seem to display strong numerical understanding and processing abilities if presented with purely mathematical problems in standard formats (Yang et al., 2024), particularly for small numbers and simple mathematical operations (of the kinds used in linguistics contest problems). Current reasoning models appear to perform well at arithmetic and algebra, math word problems (Ahn et al., 2024), and difficult mathematical contest questions equivalent to advanced college-level math problems (Fang et al., 2024; Chervonyi et al., 2025), although their problem-solving ability is sometimes inconsistent (McCoy et al., 2023; Shojaee et al., 2025). If such models are unable to solve linguistic-mathematical problems involving much simpler mathematics, and introducing linguistic structure into the problem causes their reasoning ability to break down, this indicates limitations in the *scope* of their reasoning — models may be unable to apply their reasoning flexibly across domains in the ways that humans do.

3 Methods

Models. We used OpenAI o1-mini (Jaech et al., 2024) and DeepSeek-R1-distill-Qwen-7B (Guo et al., 2025) reasoning models to conduct our experiments, querying o1-mini via the API and running DeepSeek locally. All code and data used for our experiments are available at `https://github.com/antara-raaghavi/multilingual-number-puzzles`.

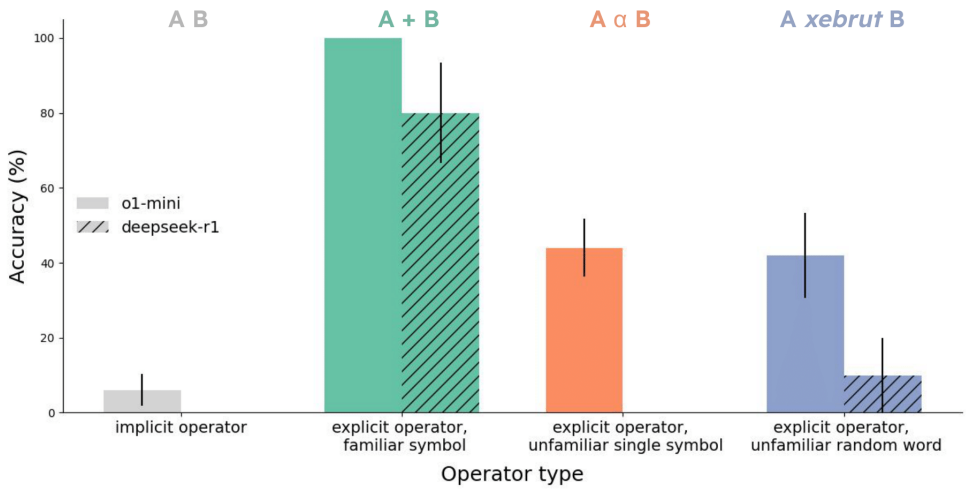

Figure 1: **Making operators explicit significantly improves performance**. Results for explicit operator experiments, for the single-character variable case (For the results on multi-character variables, see Section B Figure 7). Making operators explicit shows performance improvement over the IMPLICIT condition, but this is only substantially and reliably the case when the operator is made explicit with a familiar symbol like "+". Error bars denote standard error of the mean. 10 problems, 5 iterations per problem.

We additionally queried an instruction-tuned model (qwen-2-7b-vl-instruct) and a base model (llama-3.1-8B), both of which had an accuracy of 0 across all conditions that we test. These models almost always generated longer text answers without numbers rather than the simple numerical answer required, and were hence excluded from our analyses.

Data. We obtained data for linguistics olympiad problems from two publicly available datasets: LingOly (Bean et al., 2024) and Linguini (Sánchez et al., 2024), filtering both datasets for problems tagged as "number systems". After filtering, we had 15 problems from the LingOly and 8 problems from the Linguini dataset. Not every problem in the dataset could be standardized in the ways that our experiments required. The entire dataset was thus manually evaluated for suitable problems, and 10 problems were chosen for evaluation, all in distinct languages (see Section D). These problems spanned a range of difficulty from the first round of the UK Linguistics Olympiad to the International Linguistics Olympiad (most challenging).

4 Experiments

4.1 The effect of explicit operators in problems

Since so many of the mathematical operators in numeral structure are implicit (e.g. in English we say 'twenty three' to mean 'twenty + three'), our first experiment investigates how this implicit structure affects how models solve the problems. To do this, we standardize and convert the 10 existing linguistic number system problems to mathematical problems, and vary how explicit the operators are, as shown in Table 1.

First, we standardize all problems to control for model tokenization and task-external knowledge effects: we identify all meaningful morphemes, standardize all phonological changes, and replace them with dummy words as described in detail in Section A. This standardized version of each problem is what we call the IMPLICIT setting, since the mathematical operations are largely implicit, as they are in language. Taking these IMPLICIT problems as our baselines, we then make the operators explicit in three ways: 1) as the familiar mathematical operator symbols that perform the operation (e.g. '+' for addition), 2) as symbols that are unfamiliar for performing that operation, and 3) as whole words sampled from the tokenizer. A full example prompt with a puzzle in four variations is provided in Section B.

We present our results in Figure 1. In all cases, the presence of explicit operations with familiar symbols yields significant improvements over the default IMPLICIT condition (o1-mini performs at ceiling). In the multi-character setting (more linguistic), models perform better on average in the IMPLICIT condition than in the case with an explicit operator as an unfamiliar random word (vid. Figure 7). It is likely harder to differentiate between function words (operators) and number words (nu-